JAVA PROJECT REPORT ON

HOSPITAL BED ALLOCATION AND AVAILABILITY MONITORING SYSTEM

Submitted by

VAUMSI KRISHNA .A
25BIT161

Under the Guidance of

MUGILAN R
Technical Trainer
Technical Department, KG College of Arts and Science

Academic Year: 2025 - 2026
APRIL 2026
Semester: II

DECLARATION

I hereby declare that the project titled "**HOSPITAL BED ALLOCATION AND AVAILABILITY MONITORING SYSTEM**" is my own original work, submitted in partial fulfillment of the **Java Programming** course, under the guidance of **MUGILAN R,** Technical Trainer, KG College of Arts and Science.

This project has not been submitted anywhere else for any degree or award. All sources I used have been properly mentioned.

April 2026

Coimbatore

_____

(**VAUMSI KRISHNA .A**)

CERTIFICATE

This is to certify that the project titled "**HOSPITAL BED ALLOCATION AND AVAILABILITY MONITORING SYSTEM**" is a genuine work done by **VAUMSI KRISHNA A**, B.Sc. IT B (2025-2028), KG College of Arts and Science, as part of the **Java Programming** course.

This project was completed under my guidance during April 2026.

Place: Coimbatore
Date:

Viva-Voce Examination held on: _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page No.
1	Introduction	
	1.1 Objective	
	1.2 Overview	
	1.3 Features	
2	Program Analysis	
	2.1 Problem Definition	
	2.2 Project Overview	
	2.3 Scopes and Limitations	
3	System Design	
	3.1 Modules	
4	Methodology	
	4.1 Problem Analysis	
	4.2 System Design	
	4.3 Program Development	
	4.4 Testing and Validation	
	4.5 Result and Documentation	
	4.6 Algorithm	
	4.7 Flowchart	
5	Source Code	
6	Input and Output Screens	
7	Discussion and Conclusion	
	7.1 Discussion	
	7.2 Conclusion	
8	References	

ABSTRACT

The Hospital Bed Allocation and Availability Monitoring System is a simple Java program that helps hospitals keep track of their beds. In many hospitals, staff write down bed information in notebooks or paper files. This takes a lot of time and can lead to mistakes such as giving the same bed to two patients at the same time.

This project is a console-based application. It means the program runs in the Command Prompt window. The program lets hospital staff add bed details, admit a patient to a free bed, discharge a patient, and check which beds are free or occupied.

The program is written using Core Java. Core Java is the basic part of Java that includes simple concepts like variables, if-else conditions, loops, arrays, classes, and objects. This project is a good way to learn how these basic Java concepts can be used to make a useful real-world application.

The goal of this project is to make hospital bed management easier, faster, and more accurate for hospital staff.

1. INTRODUCTION

A hospital has many beds spread across different wards like General Ward, ICU, Maternity Ward, and Paediatric Ward. Managing these beds manually is very difficult, especially when many patients are coming in every day. Staff have to check many papers to find a free bed, which wastes time.

This project, Hospital Bed Allocation and Availability Monitoring System, solves this problem. It is a simple Java program where hospital staff can use a menu to manage beds easily from the computer. Staff can add new beds, admit patients, discharge patients, and see which beds are free, all in one place.

This project uses Core Java concepts that every beginner learns first: classes, objects, arrays, loops, and switch statements. By reading and understanding this project, a beginner can learn how these basic concepts work together to create a useful program.

1.1 Objective

The main goal of this project is to build a simple Java application that helps hospitals manage their beds without using paper.

The specific objectives are:

- To create a Java program that stores bed information.
- To allow staff to admit a patient and automatically assign a free bed.
- To allow staff to discharge a patient and free up the bed.
- To show which beds are available and which are occupied.
- To allow staff to search for a patient using their ID.
- To use basic Java concepts like classes, objects, arrays, loops, and switch statements.

1.2 Overview

When the program starts, a menu appears on the screen. The menu has 6 options:

- Option 1 - Register a bed (add a new bed to the system)
- Option 2 - Admit a patient (give a free bed to a new patient)
- Option 3 - Discharge a patient (free up the bed when patient leaves)
- Option 4 - View all beds (see which beds are free or occupied)
- Option 5 - Search a patient by ID (find which bed a patient is in)
- Option 6 - Exit the program

1.3 Features

This system has the following features:

- Bed Registration - The staff can add beds by entering a bed number and ward type.
- Patient Admission - The program finds the first available bed and assigns it to the new patient automatically.
- Discharge Management - When a patient leaves, the bed is marked as free so it can be given to someone else.
- Availability Monitoring - Displays a list of all beds with their current status (Available or Occupied).
- Patient Search - Staff can type a patient ID to find out which bed that patient is in.
- Menu-Driven Interface - All features are accessed through a simple number menu, which is easy to use.

2. PROGRAM ANALYSIS

Before writing the program, it is important to understand the problem clearly. This stage is called Program Analysis. In this stage, we study what the problem is, what the program needs to do, and what its limits are.

2.1 Problem Definition

In many hospitals, bed records are kept in registers or paper files. This creates the following problems:

- It is hard to find a free bed quickly when a patient arrives.
- Two patients may accidentally be given the same bed.
- When a patient leaves, the register may not be updated on time.
- Staff have to flip through many pages to check availability.
- There is no easy way to search for which bed a specific patient is in.

To solve these problems, we are building a simple Java program that keeps all bed information on the computer and allows staff to update it quickly through a menu.

2.2 Project Overview

This project is a console-based Java application. It uses a Bed class to store information about each bed. The main class, HospitalBedSystem, contains the menu and all the logic for admitting patients, discharging patients, and displaying bed status.

Here is what the program can do:

- Add new beds with bed number and ward type.
- Admit a patient - finds the first free bed and assigns it automatically.
- Discharge a patient - frees the bed when patient leaves.
- Show all beds with their current status.
- Search for a patient by their ID.

2.3 Scopes and Limitations

Scope

This project can be used in:

- Small hospitals and clinics
- Nursing homes
- Private medical centers
- College labs for Java practical learning

Limitations

- The data is stored in the computer's memory only. When the program closes, all data is lost because no database is used.
- Only one user can use the program at a time.
- There is no graphical interface (GUI). Everything runs in the Command Prompt.
- Advanced features like billing, reports, and online booking are not included.

3. SYSTEM DESIGN

System design means planning how the program will be structured and how different parts will work together. We divide the program into smaller parts called modules. Each module does one specific job.

3.1 Modules

This program has 6 modules:

1. Bed Registration Module

This module lets the staff add a new bed to the system. The staff enters the bed number and the ward type (for example: General, ICU). The bed is saved as Available at first.

2. Patient Admission Module

This module admits a new patient. The staff enters the patient ID and name. The program automatically finds the first available bed and assigns it to the patient. The bed status changes to Occupied.

3. Discharge Management Module

This module handles patient discharge. The staff enters the patient ID. The program finds the bed that patient is in and marks it as Available again, so another patient can use it.

4. Availability Monitoring Module

This module shows a list of all beds. For each bed, it shows the bed number, ward type, and whether it is Available or Occupied. If occupied, it also shows the patient name.

5. Search Module

This module lets staff search for a patient by entering their ID. The program then shows the bed number and ward where that patient is admitted.

6. Exit Module

This module closes the program when the user selects option 6 from the menu.

4. METHODOLOGY

Methodology means the step-by-step process followed to build the project. We start from understanding the problem and go all the way to testing the final program.

4.1 Problem Analysis

First, we studied the problems that hospital staff face when managing beds manually. The main problems are:

- No quick way to see which beds are free.
- Risk of assigning the same bed to two patients.
- Slow process to update records when patients are discharged.

After understanding these problems, we decided to build a Java program that handles all of this automatically.

4.2 System Design

Next, we designed the structure of the program. We decided to use:

- A Bed class to store bed details such as bed number, ward, status, and patient info.
- An array of Bed objects to store up to 20 beds.
- A do-while loop for the menu so it keeps showing until the user chooses to exit.
- A switch statement to handle each menu option.

4.3 Program Development

We wrote the Java program step by step:

- First, we created the Bed class with all required fields and a display method.
- Then, we wrote the main method with the menu loop.
- We added code for each menu option one by one (register, admit, discharge, view, search).
- We used Scanner to accept input from the user through the keyboard.

Java concepts used in this project:

- Class and Object - We created a Bed class and made Bed objects to store bed data.
- Array - We used an array to store multiple Bed objects.
- Loop - We used a do-while loop to keep the menu running.
- Switch Statement - We used switch to handle different menu choices.
- Boolean - We used true and false to track if a bed is occupied or free.

- Scanner - We used Scanner class to read input from the user.

4.4 Testing and Validation

After writing the program, we tested it by trying each menu option to make sure it works correctly.

What We Tested	Expected Result
Register a new bed	Bed is added and shows as Available
Admit a patient to a free bed	Bed becomes Occupied, patient name is saved
Admit patient when no beds are free	Message: No beds available
Discharge a patient	Bed becomes Available again
View all beds	All beds listed with their current status
Search patient by ID	Correct bed details are displayed
Search for an ID that does not exist	Message: Patient not found

4.5 Result and Documentation

After testing, all the features worked correctly. The program successfully registers beds, admits patients, discharges them, and shows availability. The screenshots of the program running are shown in Section 6.

4.6 Algorithm

An algorithm is a list of steps that the program follows. Think of it like instructions in a recipe.

Algorithm for the Hospital Bed Allocation System:

1. Start the program.
2. Show the main menu on screen.
3. Ask the user to enter a number from 1 to 6.
4. If the user enters 1: Ask for bed number and ward type. Save the bed as Available.
5. If the user enters 2: Ask for patient ID and name. Find the first Available bed. Mark it as Occupied and save the patient details.
6. If the user enters 3: Ask for patient ID. Find the bed with that patient. Mark the bed as Available. Remove patient details.
7. If the user enters 4: Show all beds with their status (Available or Occupied).

8. If the user enters 5: Ask for patient ID. Search and show the bed details for that patient.
9. If the user enters 6: Print Exiting the system and stop.
10. If the user enters anything else: Print Invalid choice.
11. Go back to Step 2 unless the user chose 6.
12. End.

4.7 Flowchart

The flowchart below describes the flow of the program from start to finish:

```
          [START]
             |
    [Show Main Menu]
             |
   [User Enters Choice]
             |
 __________|__________
  |    |    |    |    |
 [1]  [2]  [3]  [4]  [5]  [6]
 Add  Admit Disc View Search Exit
 Bed  Pt    Pt  Beds  by ID
  |    |    |    |    |    |
 [Do the action and show result]
             |
    [Return to Menu]
             |
    [Choice == 6?]
     YES         NO
     |            |
   [END]    [Show Menu Again]
```

5. SOURCE CODE

Below is the complete Java source code. Comments (lines starting with //) are added to explain each part so a beginner can understand easily.

File Name: HospitalBedSystem.java

```
import java.util.Scanner;
class Bed {

    int bedNumber;
    String wardType;
    boolean isOccupied;
    String patientName;
    int patientId;


    Bed(int bedNumber, String wardType) {
        this.bedNumber = bedNumber;
        this.wardType = wardType;
        this.isOccupied = false;
        this.patientName = "";
        this.patientId = 0;
    }

        void displayStatus() {
        String status;
        if (isOccupied == true) {
            status = "Occupied by: " + patientName;
        } else {
            status = "Available";
        }
        System.out.println("Bed No: " + bedNumber +
                           " | Ward: " + wardType +
                           " | Status: " + status);
    }
}

public class HospitalBedSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Array to hold up to 20 Bed objects
        Bed[] beds = new Bed[20];

        int bedCount = 0;  // counts how many beds are added
        int choice;        // stores the menu number chosen by user
```

```
        do {
            System.out.println("");
            System.out.println("--- Hospital Bed Allocation System ---");
            System.out.println("1. Register Bed");
            System.out.println("2. Admit Patient");
            System.out.println("3. Discharge Patient");
            System.out.println("4. View Bed Availability");
            System.out.println("5. Search Patient by ID");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

                        switch (choice) {

                case 1:  // Register a new bed
                    System.out.print("Enter Bed Number: ");
                    int bedNo = sc.nextInt();
                    sc.nextLine();  // clears the Enter key from memory
                    System.out.print("Enter Ward Type
(General/ICU/Maternity/Paediatric): ");
                    String ward = sc.nextLine();
                    beds[bedCount] = new Bed(bedNo, ward);  // create new
Bed object
                    bedCount++;  // increase the count
                    System.out.println("Bed registered successfully.");
                    break;

                case 2:  // Admit a new patient
                    System.out.print("Enter Patient ID: ");
                    int pid = sc.nextInt();
                    sc.nextLine();
                    System.out.print("Enter Patient Name: ");
                    String pname = sc.nextLine();
                    boolean allocated = false;
                    // Loop through all beds to find a free one
                    for (int i = 0; i < bedCount; i++) {
                        if (beds[i].isOccupied == false) {  // if bed is
free
                            beds[i].isOccupied = true;       // mark as
occupied
                            beds[i].patientName = pname;     // save
patient name
                            beds[i].patientId = pid;          // save
patient ID
                            System.out.println("Patient admitted. Bed " +
beds[i].bedNumber + " assigned.");
                            allocated = true;
                            break;  // stop searching once a bed is found
                        }
                    }
                    if (allocated == false) {
                        System.out.println("Sorry, no beds are available
right now.");
                    }
                    break;
```

```
                case 3:  // Discharge a patient
                    System.out.print("Enter Patient ID to discharge: ");
                    int dId = sc.nextInt();
                    boolean discharged = false;
                    for (int i = 0; i < bedCount; i++) {
                        if (beds[i].patientId == dId) {      // find the
patient
                            beds[i].isOccupied = false;       // mark bed
as free
                            beds[i].patientName = "";         // clear
patient name
                            beds[i].patientId = 0;             // reset
patient ID
                            System.out.println("Patient discharged. Bed " +
beds[i].bedNumber + " is now Available.");
                            discharged = true;
                            break;
                        }
                    }
                    if (discharged == false) {
                        System.out.println("Patient ID not found.");
                    }
                    break;

                case 4:  // View all bed statuses
                    if (bedCount == 0) {
                        System.out.println("No beds have been registered
yet.");
                    } else {
                        System.out.println("\nBed Availability Status:");
                        for (int i = 0; i < bedCount; i++) {
                            beds[i].displayStatus();  // call the display
method
                        }
                    }
                    break;

                case 5:  // Search for a patient by their ID
                    System.out.print("Enter Patient ID to search: ");
                    int sId = sc.nextInt();
                    boolean found = false;
                    for (int i = 0; i < bedCount; i++) {
                        if (beds[i].patientId == sId) {
                            beds[i].displayStatus();
                            found = true;
                        }
                    }
                    if (found == false) {
                        System.out.println("No patient found with that
ID.");
                    }
                    break;

                case 6:  // Exit the program
                    System.out.println("Exiting the system. Goodbye!");
                    break;
```

```
                default:
                    System.out.println("Invalid choice. Please enter a
number from 1 to 6.");
            }

        } while (choice != 6);  // loop stops when user types 6

        sc.close();  // always close Scanner when done
    }
}
```

6. INPUT AND OUTPUT SCREENS

INPUT SCREEN:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\java practical\project>javac HospitalBedSystem.java

D:\java practical\project>java HospitalBedSystem
```

OUTPUT SCREEN:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\java practical\project>javac HospitalBedSystem.java

D:\java practical\project>java HospitalBedSystem

--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
```

FINAL OUTPUT SCREEN:

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\java practical\project>javac HospitalBedSystem.java

D:\java practical\project>java HospitalBedSystem

--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
Enter your choice: 1
Enter Bed Number: 2
Enter Ward Type (General/ICU/Maternity/Paediatric): General
Bed registered successfully.

--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
Enter your choice: 4

Bed Availability Status:
Bed No: 2 | Ward: General | Status: Available

--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
Enter your choice: 3
Enter Patient ID to discharge: 1
Patient ID not found.
```

C:\Windows\System32\cmd.e

```
--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
Enter your choice: 3
Enter Patient ID to discharge: 1
Patient ID not found.

--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
Enter your choice: 3
Enter Patient ID to discharge: 2
Patient ID not found.

--- Hospital Bed Allocation System ---
1. Register Bed
2. Admit Patient
3. Discharge Patient
4. View Bed Availability
5. Search Patient by ID
6. Exit
Enter your choice: 6
Exiting the system. Goodbye!

D:\java practical\project>
```

7. DISCUSSION AND CONCLUSION

7.1 Discussion

This project showed how simple Java concepts can be used to build a useful real-world application. A beginner who knows classes, objects, arrays, loops, and switch statements can fully understand and build this program.

The most important Java concepts used in this project are:

- Class and Object: The Bed class stores all data for one bed. We created many Bed objects using an array.
- Array: We used an array called beds[] to store up to 20 Bed objects.
- Boolean: We used a boolean variable isOccupied to track whether a bed is free or not.
- Loop (for and do-while): The for loop searches through beds, and the do-while loop keeps the menu running.
- Switch Statement: The switch handles which action to perform based on the user's menu choice.
- Scanner: We used Scanner to read input (numbers and text) from the user via the keyboard.

During testing, all six menu options worked correctly. The program correctly prevented admitting to occupied beds and showed appropriate messages when patients were not found.

7.2 Conclusion

The Hospital Bed Allocation and Availability Monitoring System was successfully built using Core Java. The program helps hospital staff manage beds easily without using paper registers. It can add beds, admit patients, discharge patients, view availability, and search for patients.

This project is a good example for Java beginners because it uses only basic concepts. After understanding this project, a student can move forward to learn more advanced topics.

Future improvements that can be added later:

- Connect the program to a database like MySQL to save data permanently even after the program closes.
- Build a Graphical User Interface (GUI) using Java Swing so it has buttons and windows instead of text menus.
- Add login and password protection for hospital staff.
- Add a feature to generate a daily report of bed occupancy.

8. REFERENCES

Books:

- Herbert Schildt - Java: The Complete Reference (A very good book for learning Java from basics to advanced topics)

Websites:

- https://www.oracle.com/java (Official Java documentation and tutorials)
- https://www.geeksforgeeks.org (Simple Java examples with explanations for beginners)
- https://www.w3schools.com (Easy-to-understand Java tutorials with Try It examples)
- https://www.javatpoint.com (Beginner-friendly Java notes and programs)